

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №4

По дисциплине

“Основы программной инженерии”

Вариант: 39493

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р3413

Преподаватель:
Обляшевский Севастьян Владиславович

Санкт-Петербург, 2025 г

Текст задания

1. Для своей программы из лабораторной работы #3 по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий средний интервал между кликами пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить значение переменной classpath для данной JVM.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя потока, потребляющего наибольший процент времени CPU.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Отчёт по работе должен содержать:

Текст задания.

1. Исходный код разработанных MBean-классов и сопутствующих классов.
2. Скриншоты программы JConsole со снятыми показаниями, выводы по результатам мониторинга.
3. Скриншоты программы VisualVM со снятыми показаниями, выводы по результатам профилирования.
4. Скриншоты программы VisualVM с комментариями по ходу поиска утечки памяти.
5. Выводы по работе.

Ход выполнения

Исходный код 3 лабораторной работы по дисциплине «Веб-программирование» - <https://github.com/stoneshik/third-lab-web>

Исходный код данной лабораторной — <https://github.com/stoneshik/opi-fourth-lab>

1) Исходный код разработанных MBean-классов и сопутствующих классов

Интерфейс первого MBean, считающего общее число установленных пользователем точек, а также число точек, не попадающих в область:

```
package lab.third.jmx;
```

```
public interface DotStatsMBean {  
    /** Общее число установленных пользователем точек */  
    long getTotalCount();  
    /** Число попавших точек */  
    long getHitCount();  
    /** Число промахов */  
    long getMissCount();  
}
```

Класс реализации первого MBean:

```
package lab.third.jmx;
```

```
import javax.management.Notification;
```

```
import javax.management.NotificationBroadcasterSupport;
```

```
public class DotStats extends NotificationBroadcasterSupport implements DotStatsMBean {  
    private static final String MISS_NOTIFICATION_TYPE = "lab.third.miss";  
    private long totalCount;  
    private long hitCount;  
    private long missCount;  
    private long sequenceNumber;
```

```
    /**  
     * Регистрирует попадание или промах  
     */  
    public synchronized void registerDot(boolean hit) {  
        totalCount++;  
        if (hit) {  
            hitCount++;  
        } else {  
            missCount++;  
            sendMissNotification();  
        }  
    }  
}
```

```
    private void sendMissNotification() {  
        Notification notification = new Notification(  
            MISS_NOTIFICATION_TYPE,  
            this,  
            ++sequenceNumber,  
            System.currentTimeMillis(),  
            "Miss № " + missCount  
        );  
        sendNotification(notification);  
    }  
}
```

```
@Override  
public synchronized long getTotalCount() { return totalCount; }  
@Override  
public synchronized long getHitCount() { return hitCount; }  
@Override  
public synchronized long getMissCount() { return missCount; }
```

```
}
```

Интерфейс второго MBean, определяющий средний интервал между кликами пользователя по координатной плоскости:

```
package lab.third.jmx;
```

```
public interface ClickIntervalMBean {  
    /** Средний интервал между кликами (мс) */  
    double getAverageIntervalMillis();  
    /** Количество зарегистрированных кликов */  
    long getClickCount();  
    /** Сброс статистики */  
    void reset();  
}
```

Класс реализации второго MBean:

```
package lab.third.jmx;
```

```
public class ClickInterval implements ClickIntervalMBean {  
    private long lastClickTime = -1;  
    private long totalInterval = 0;  
    private long intervalCount = 0;  
    private long clickCount = 0;  
  
    /**  
     * Вызывать при каждом клике пользователя  
     */  
    public synchronized void registerClick() {  
        long now = System.currentTimeMillis();  
        clickCount++;  
        if (lastClickTime != -1) {  
            long interval = now - lastClickTime;  
            totalInterval += interval;  
            intervalCount++;  
        }  
        lastClickTime = now;  
    }  
  
    @Override  
    public synchronized double getAverageIntervalMillis() {  
        if (intervalCount == 0) {  
            return 0.0;  
        }  
        return (double) totalInterval / intervalCount;  
    }  
  
    @Override  
    public synchronized long getClickCount() { return clickCount; }  
  
    @Override  
    public synchronized void reset() {  
        lastClickTime = -1;  
        totalInterval = 0;  
        intervalCount = 0;  
        clickCount = 0;  
    }  
}
```

Класс регистрации MBean:

```
package lab.third.jmx;

import javax.management.MBeanServer;
import javax.management.ObjectName;
import java.lang.management.ManagementFactory;

public final class JmxRegistry {
    private static final DotStats DOT_STATS = new DotStats();
    private static final ClickInterval CLICK_INTERVAL = new ClickInterval();
    static {
        try {
            MBeanServer server = ManagementFactory.getPlatformMBeanServer();
            server.registerMBean(
                DOT_STATS,
                new ObjectName("lab.third:type=DotStats")
            );
            server.registerMBean(
                CLICK_INTERVAL,
                new ObjectName("lab.third:type=ClickInterval")
            );
        } catch (Exception e) {
            throw new IllegalStateException("JMX MBean registration failed", e);
        }
    }

    private JmxRegistry() {}
    public static DotStats dotStats() { return DOT_STATS; }
    public static ClickInterval clickInterval() { return CLICK_INTERVAL; }
}
```

DotManagedBean с интеграцией Mbean-классов:

```
package lab.third.beans;

import java.io.Serializable;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

import jakarta.enterprise.context.ApplicationScoped;
import jakarta.enterprise.inject.Model;
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import lab.third.entities.DotEntity;
import lab.third.jmx.JmxRegistry;
import lab.third.util.TransactionExecutor;

@Model
@ApplicationScoped
public class DotManagedBean implements Serializable {
    private static final long serialVersionUID = 1L;
    private final EntityManagerFactory entityManagerFactory = Persistence.createEntityManagerFactory(
        "opiPU"
    );
    private DotEntity currentEntity = new DotEntity();
    private List<DotEntity> dotEntities = new CopyOnWriteArrayList<>();

    public DotManagedBean() {
        initTransaction(
            manager -> this.dotEntities.addAll(
                manager.createQuery(
                    "SELECT e FROM DotEntity e",
                    DotEntity.class
                ).getResultList()
            )
        );
    }
}
```

```

public DotEntity getCurrentEntity() {
    return currentEntity;
}

public List<DotEntity> getDotEntities() {
    return dotEntities;
}

public void setCurrentEntity(DotEntity newDotEntity) {
    this.currentEntity = newDotEntity;
}

public void setDotEntities(List<DotEntity> dotEntities) {
    this.dotEntities = dotEntities;
}

public boolean isEmpty() {
    return this.dotEntities.isEmpty();
}

public synchronized void addNewDot() {
    long startTime = System.nanoTime();
    currentEntity.updateInfo();
    currentEntity.setTimeLead(System.nanoTime() - startTime);
    // JMX: факт клика
    JmxRegistry.clickInterval().registerClick();
    // JMX: статистика попаданий
    JmxRegistry.dotStats().registerDot(currentEntity.getIsHit());
    initTransaction(manager -> manager.persist(currentEntity));
    dotEntities.add(0, currentEntity);
    currentEntity = new DotEntity();
}

public synchronized void removeAllDots() {
    initTransaction(
        manager -> manager.createQuery("DELETE FROM DotEntity").executeUpdate()
    );
    this.dotEntities.clear();
}

public void initTransaction(TransactionExecutor transactionExecutor) {
    EntityManager manager = this.entityManagerFactory.createEntityManager();
    try {
        manager.getTransaction().begin();
        transactionExecutor.executeTransaction(manager);
        manager.getTransaction().commit();
    } catch (Exception ex) {
        if (manager.getTransaction().isActive()) {
            manager.getTransaction().rollback();
        }
        System.out.println("An exception occurred during transaction.");
        ex.printStackTrace();
    } finally {
        manager.close();
    }
}
}

```

2) Проведение мониторинга программы, с помощью утилиты JConsole:

Страница запущенного веб-сервиса представлена на рисунке 1.

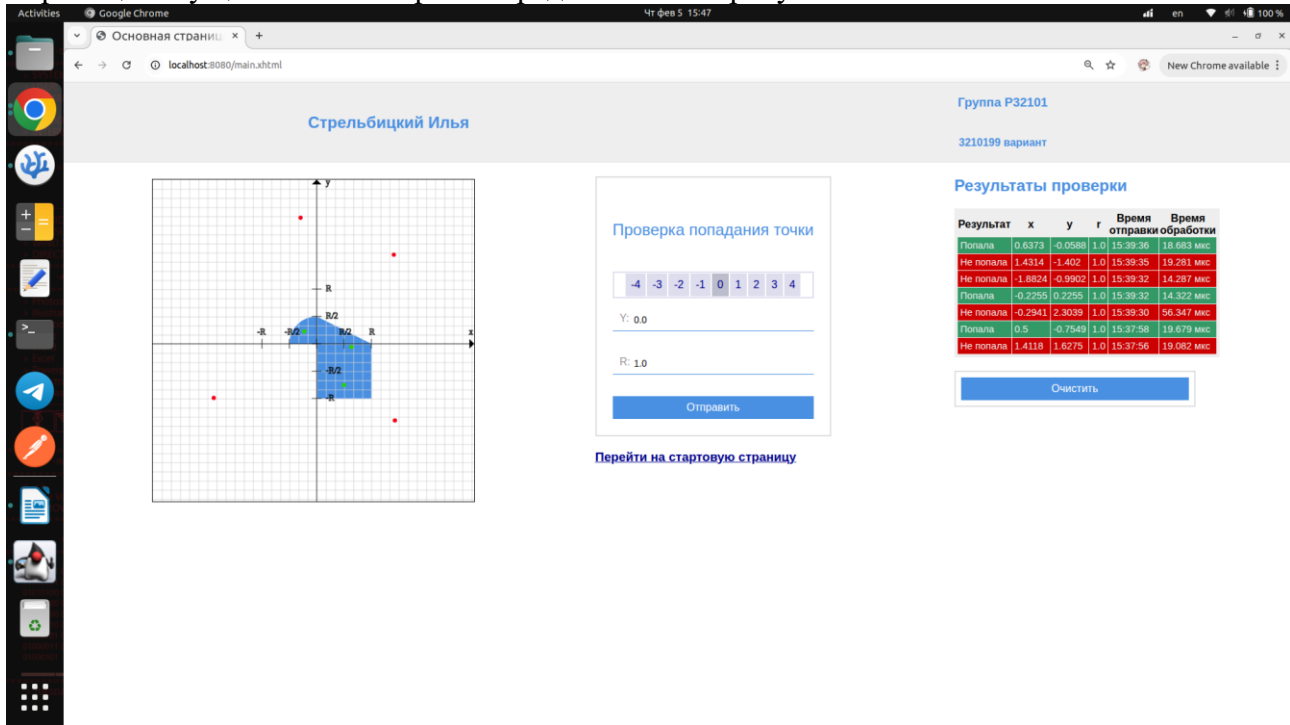


Рисунок 1. Страница запущенного веб-сервиса

Информация о первом MBean, представлена на рисунках 2 и 3.

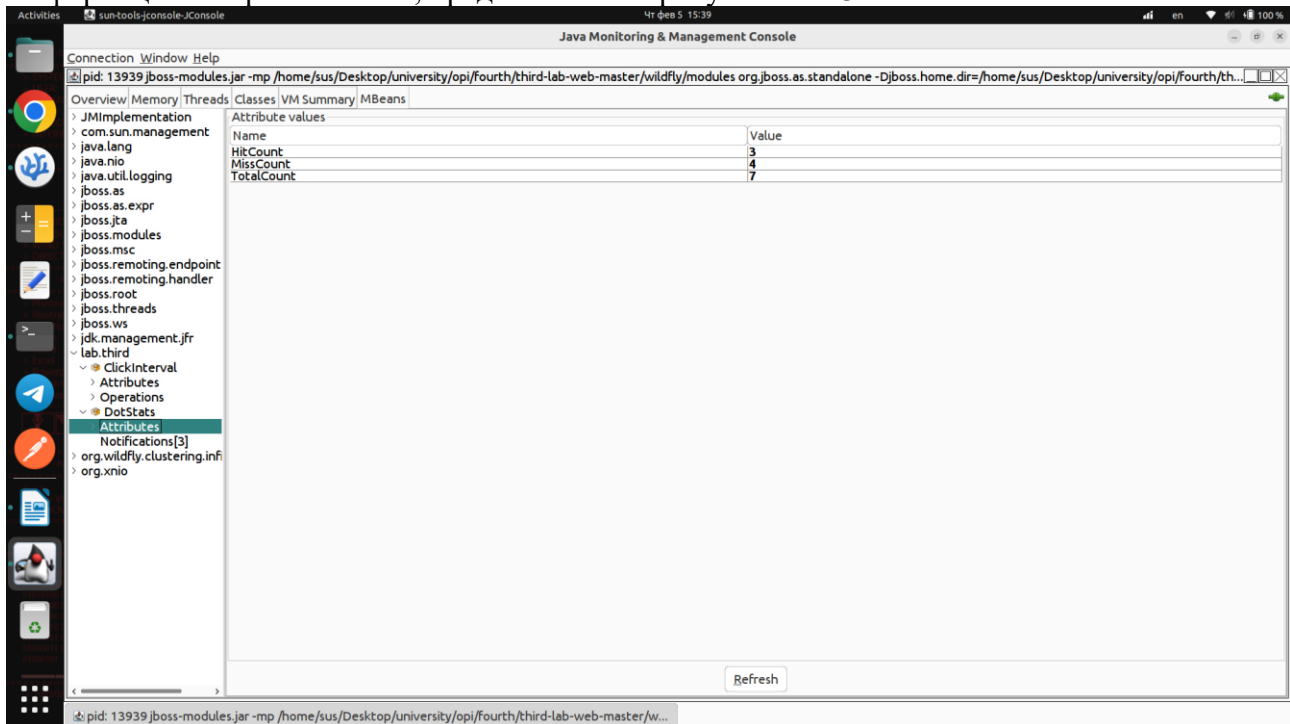


Рисунок 2. Общее количество поставленных точек, попавших и не попавших точек.

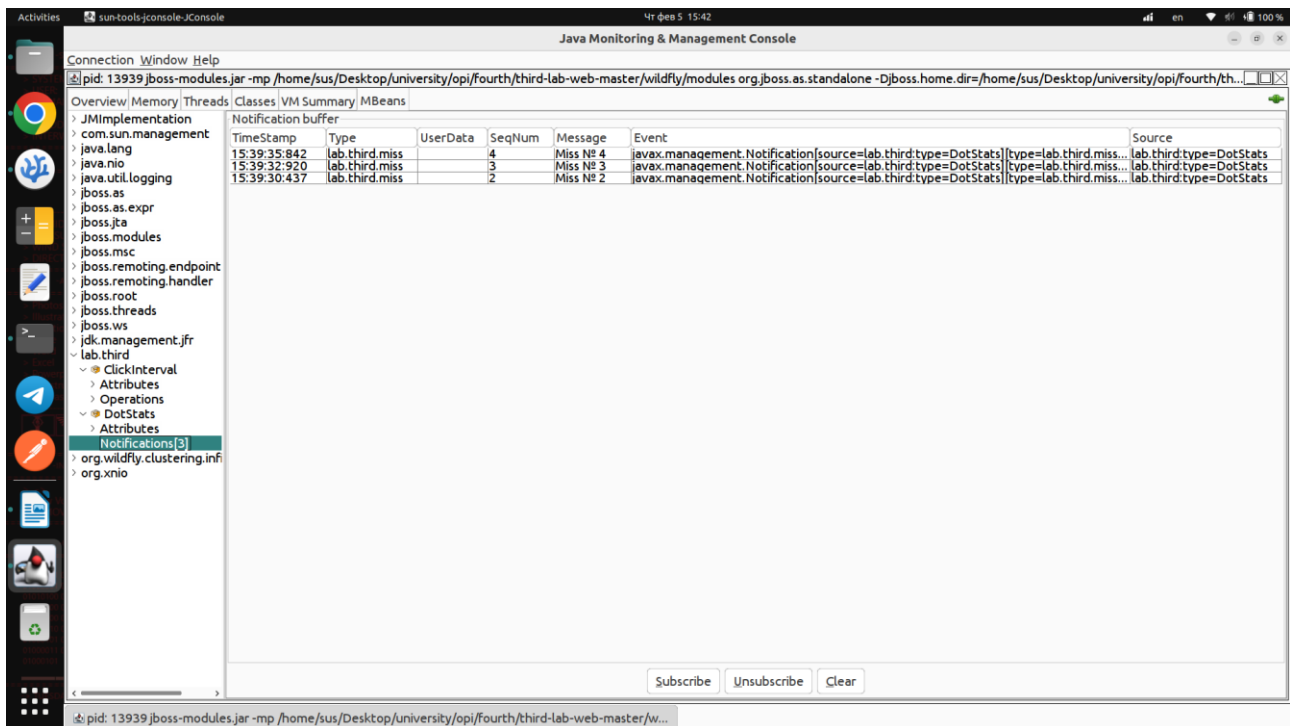


Рисунок 3. Полученные уведомления о не попавших точках (число уведомлений отличается от общего числа не попавших точек, так как не сразу было прожато subscribe).

Информация о втором MBean, представлена на рисунке 4.

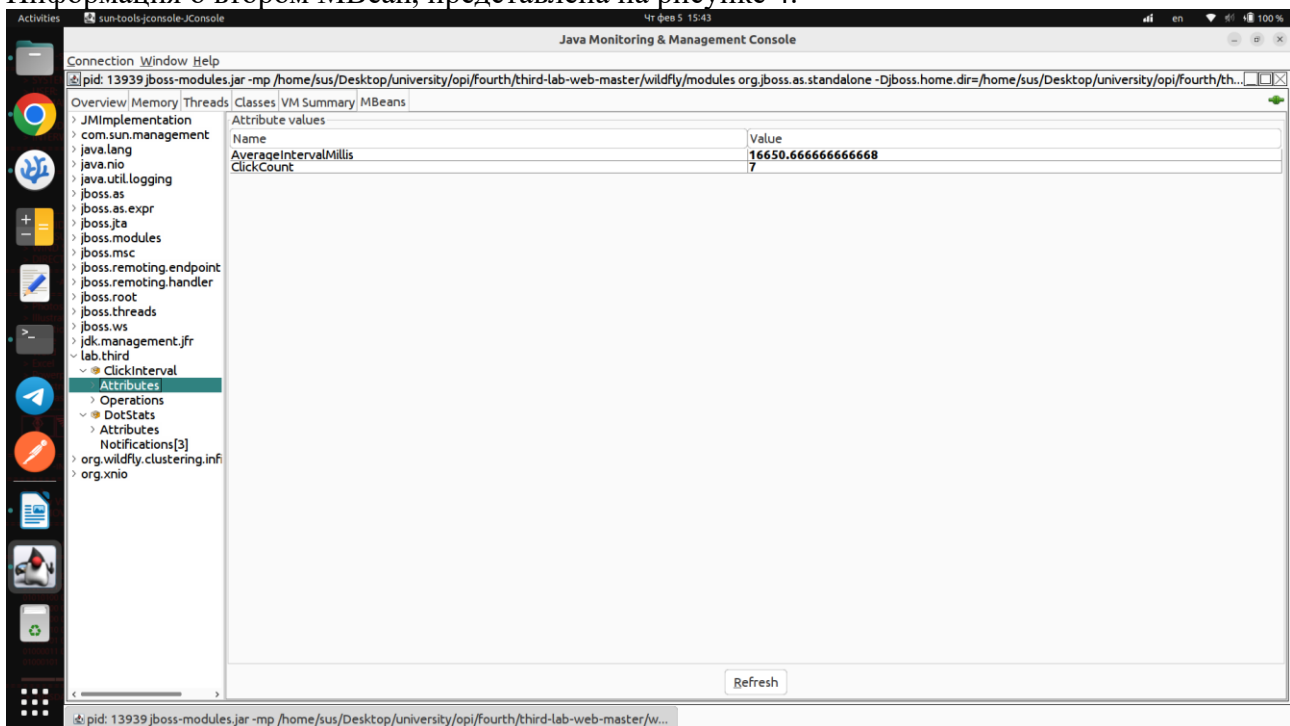


Рисунок 4. Общее число кликов и среднее время между ними.

Также во втором MBean доступна операция сброса статистики reset(). Ее запуск представлен на рисунке 5, на рисунке 6 показывается сброшенная статистика второго MBean.

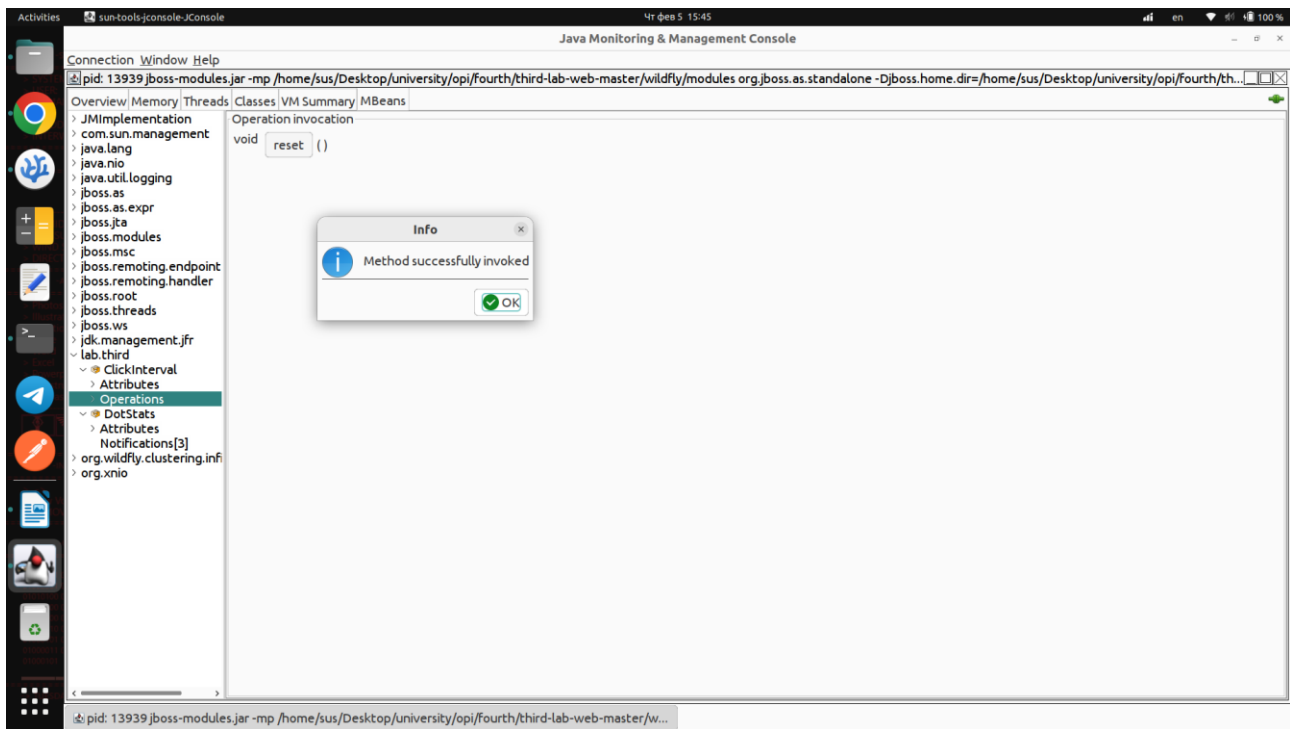


Рисунок 4. Успешное выполнение операции reset()

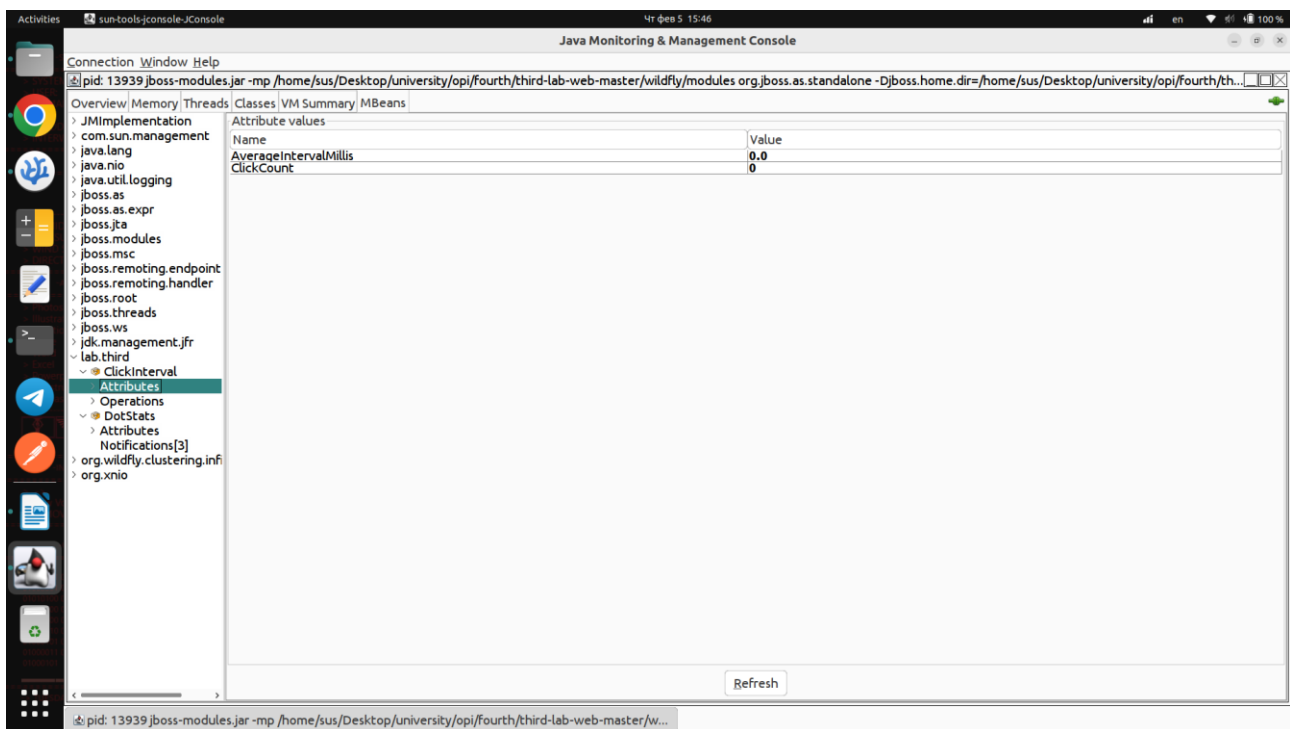


Рисунок 5. Результат выполненной операции reset() - сброшенная статистика

Далее определяем значение classpath для данной JVM. Его можно увидеть на рисунке 6, оно равняется «/home/sus/Desktop/university/opi/fourth/third-lab-web-master/wildfly/jboss-modules.jar».

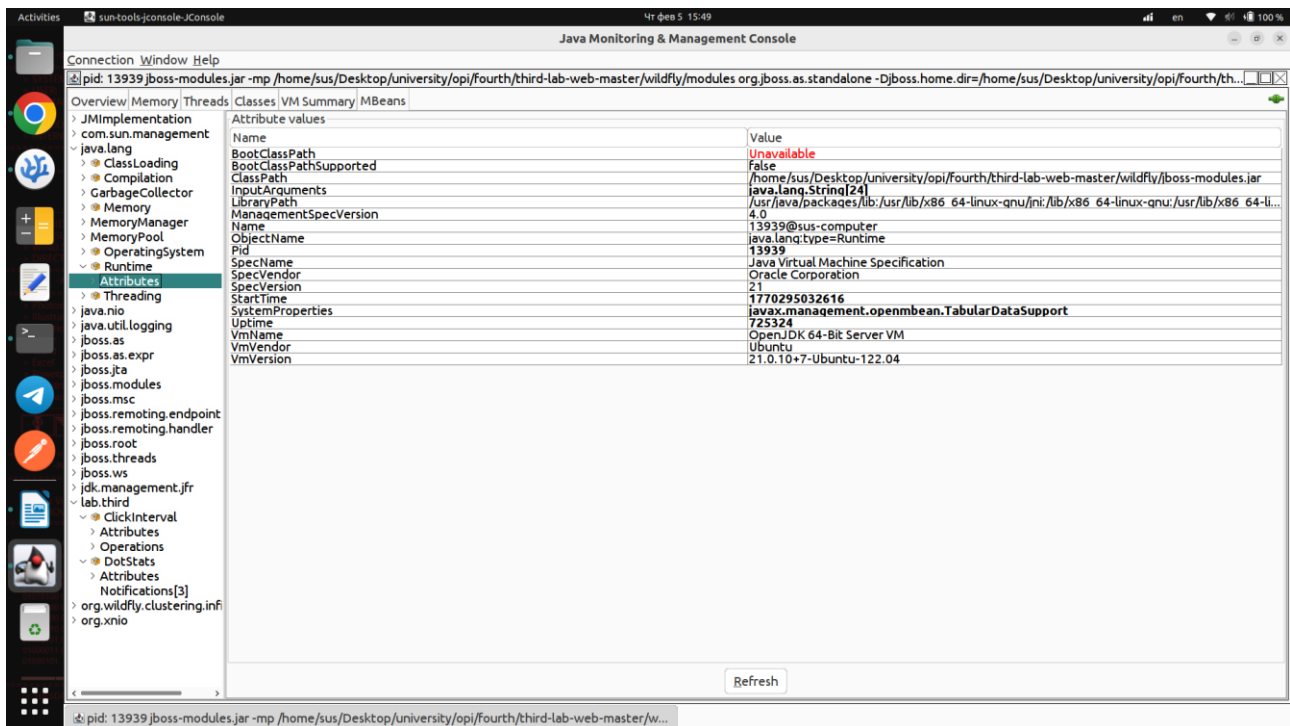


Рисунок 6. Определение значения classpath для JVM веб-сервиса

3) Проведение с помощью утилиты VisualVM мониторинга и профилирования программы:

Для начала требуется установить плагин VisualVM-MBeans в VisualVM — рисунок 7.

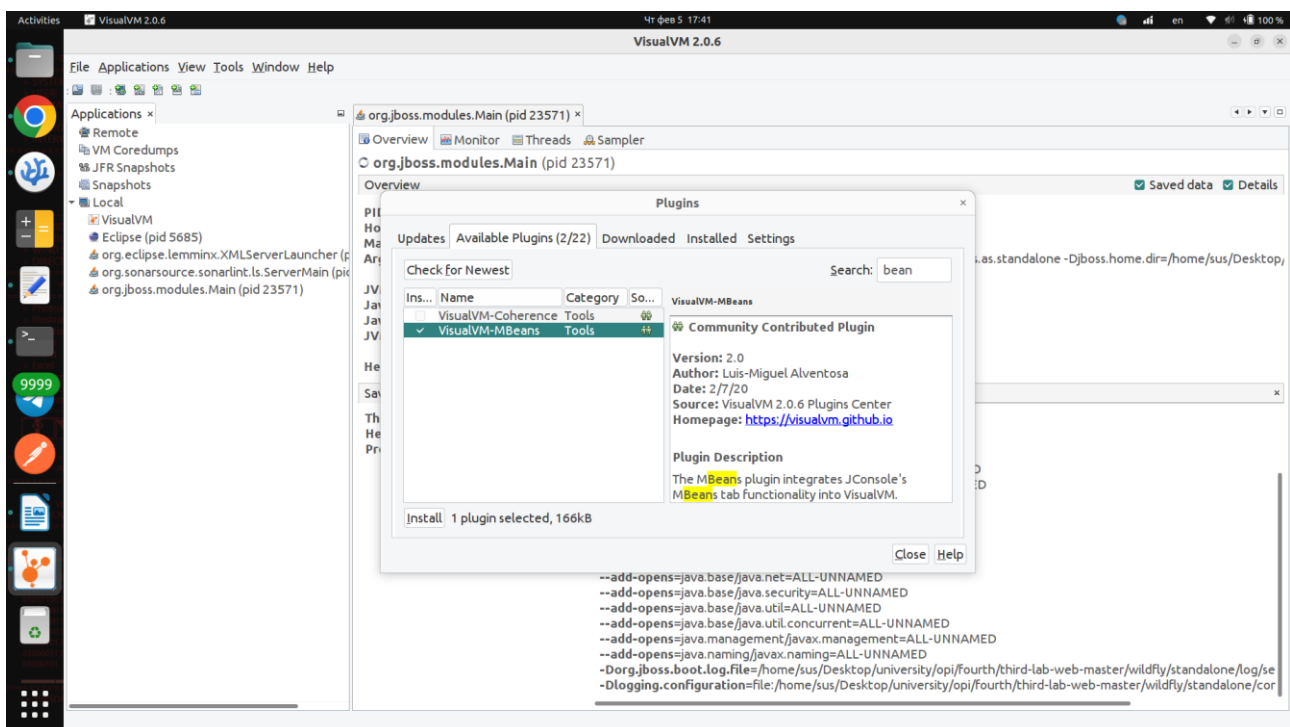


Рисунок 7. Установка плагина VisualVM-MBeans

Для того чтобы wildfly был доступен для мониторинга в VisualVM, требуется его запускать со следующими параметрами:

```
./wildfly/bin/standalone.sh -c standalone.xml -b 0.0.0.0 -bmanagement 0.0.0.0
```

Общую статистику потребления ресурсов можно увидеть на рисунке 8.

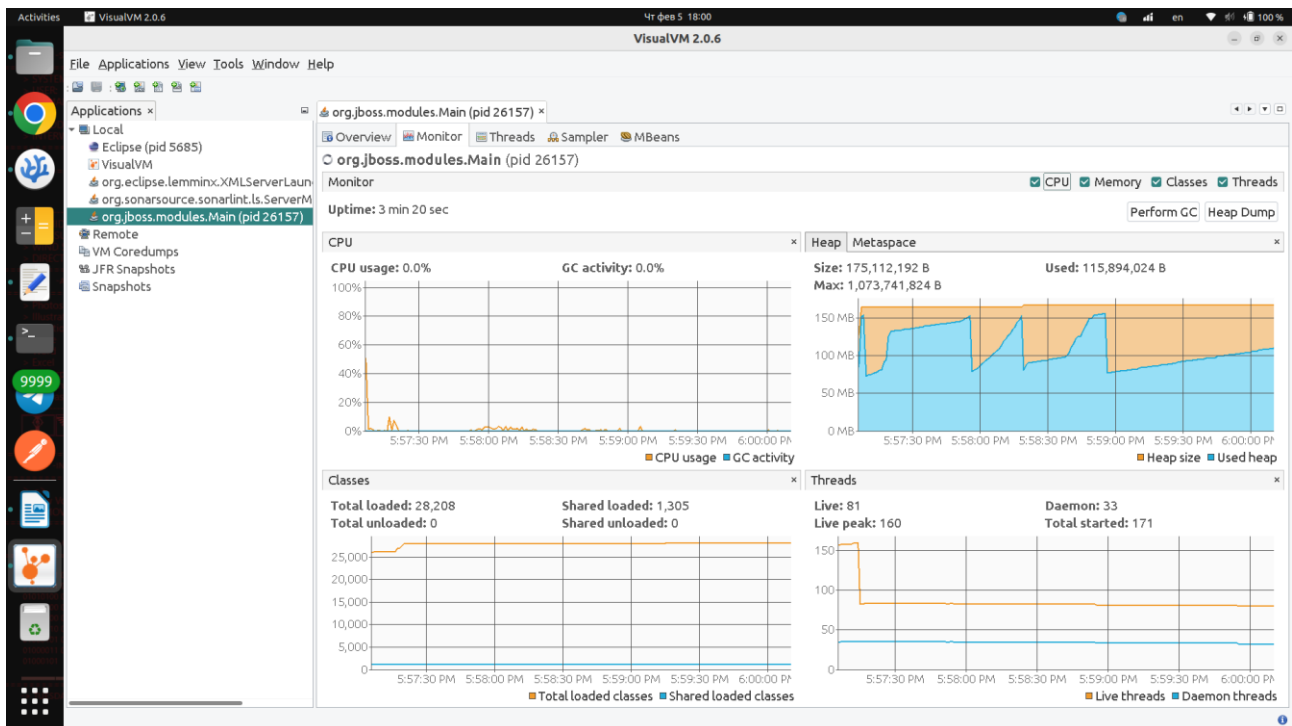


Рисунок 8. Общая статистика потребления ресурсов

Графики показаний MBean-классов, разработанных в в ходе выполнения задания 1, с течением времени можно увидеть на рисунках 9 (первый MBean) и 10 (второй MBean).

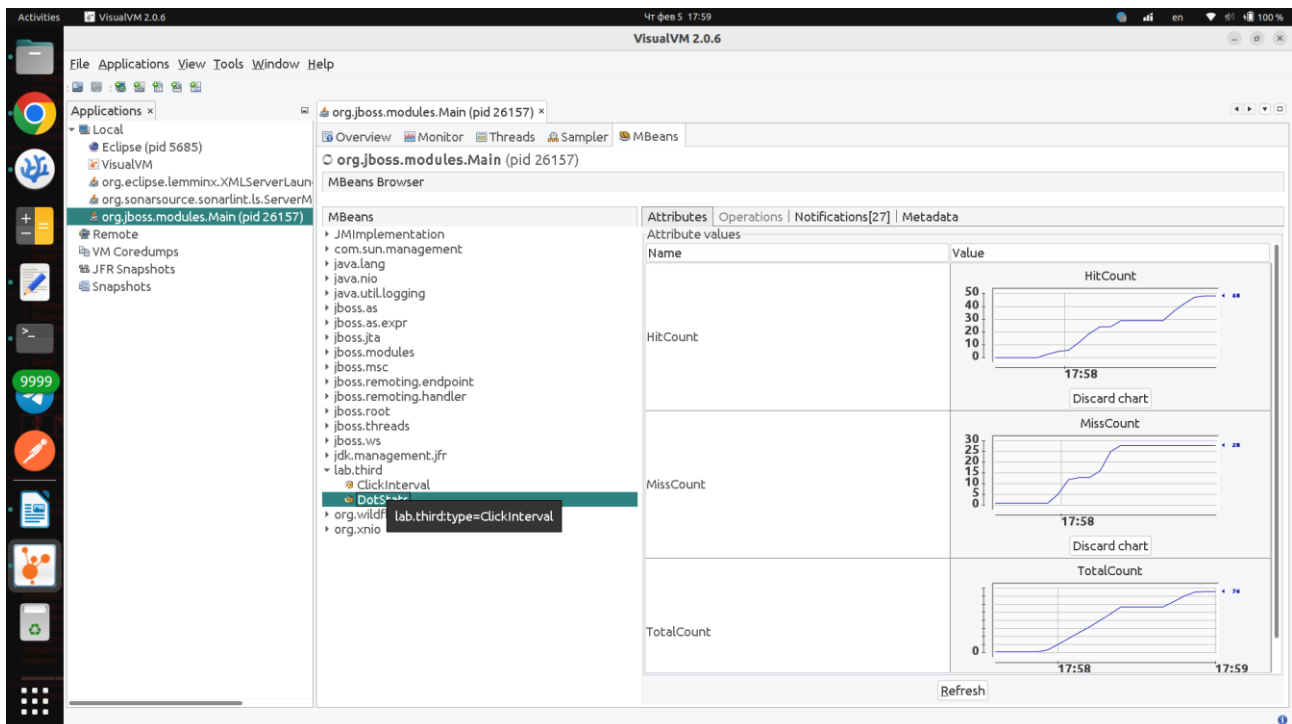


Рисунок 9. Графики показаний первого MBean-класса

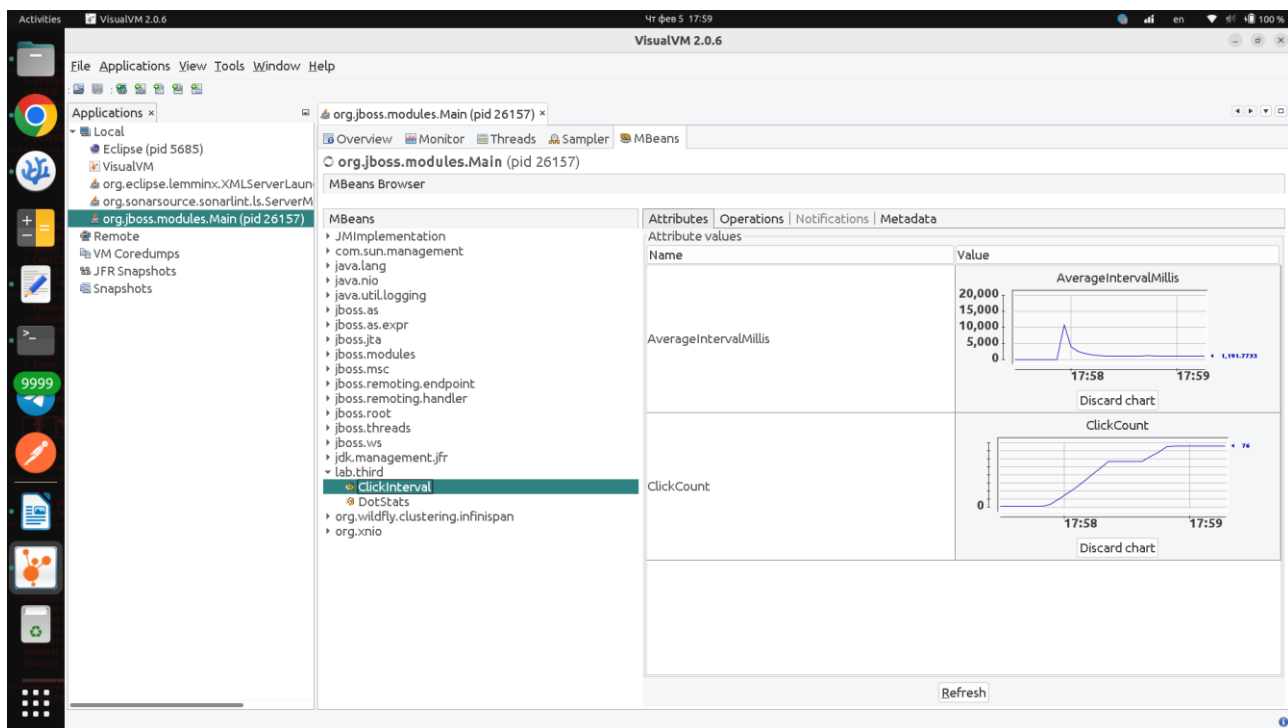


Рисунок 10. Графики показаний второго MBean-класса

Для того чтобы определить имя потока, потребляющего наибольший процент времени CPU, посмотрим Сэмпл для CPU — рисунок 11. Из него можно увидеть, что имя потока — RMI TCP Connection(4)-127.0.0.1

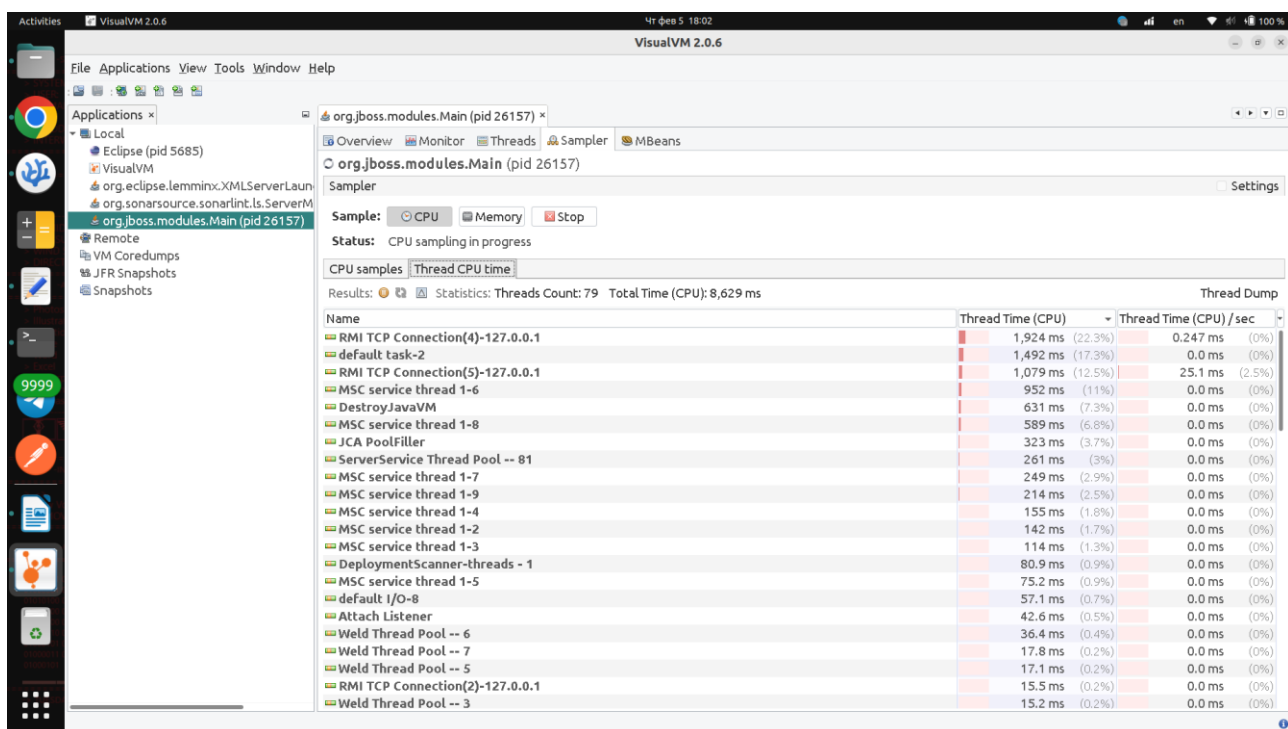


Рисунок 11. Сэмпл для CPU

4) С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

Ссылка на программу, которую будем анализировать —

<https://se.ifmo.ru/documents/10180/189115/HttpUnit.tar.gz/7bf1032e-d16e-be85-c71b-dbe73c0178ba?t=1651168887037&download=true>

В процессе анализа исходного кода программы, обнаружен ненужный вызов метода `java.lang.Thread.sleep(200)`, он приостанавливает выполнение потока на 200 миллисекунд, что приводит к ненужным задержкам в системе. Без него система работает корректно, поэтому он не несет функционального смысла, так что лучше его удалить — рисунок 12.

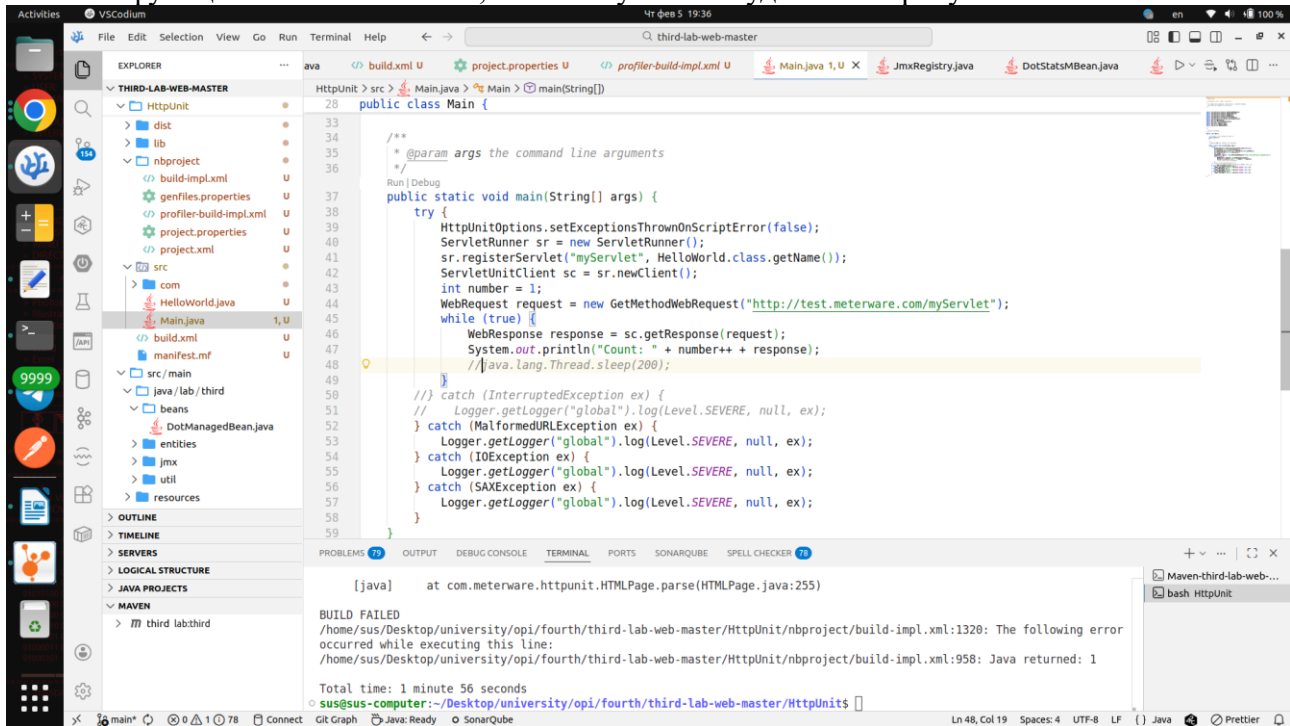


Рисунок 12. Удаление ненужной задержки.

В `.properties` для Ant у проекта установлено максимальное значение оперативной памяти в 12 Мб при помощи флага `-Xmx12m`.

Программа быстро падает, из-за того что у нее заканчивается память, вывод в терминал программы при падении представлен на рисунке 13.

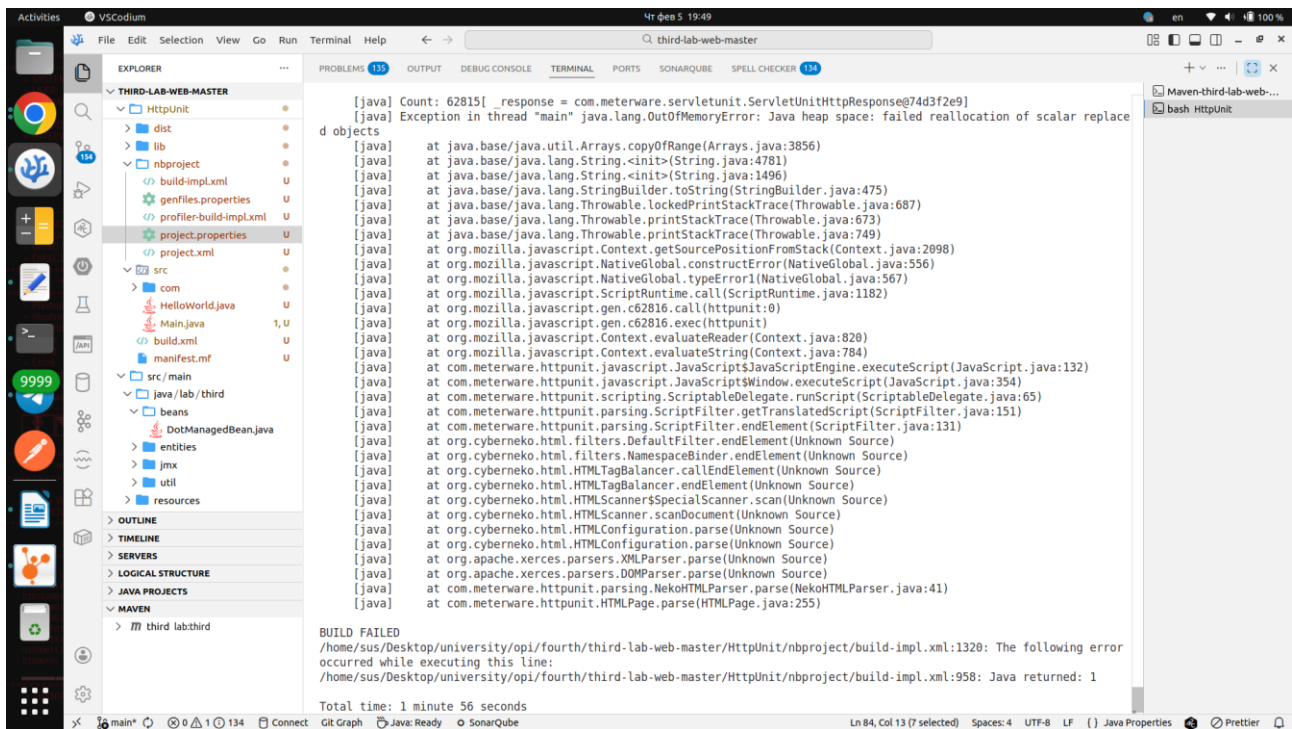


Рисунок 13. Вывод в терминал программы при падении.

Для поиска утечки памяти повысим максимальное количество памяти до 50 Мб — добавление нужного флага можно увидеть на рисунке 14.

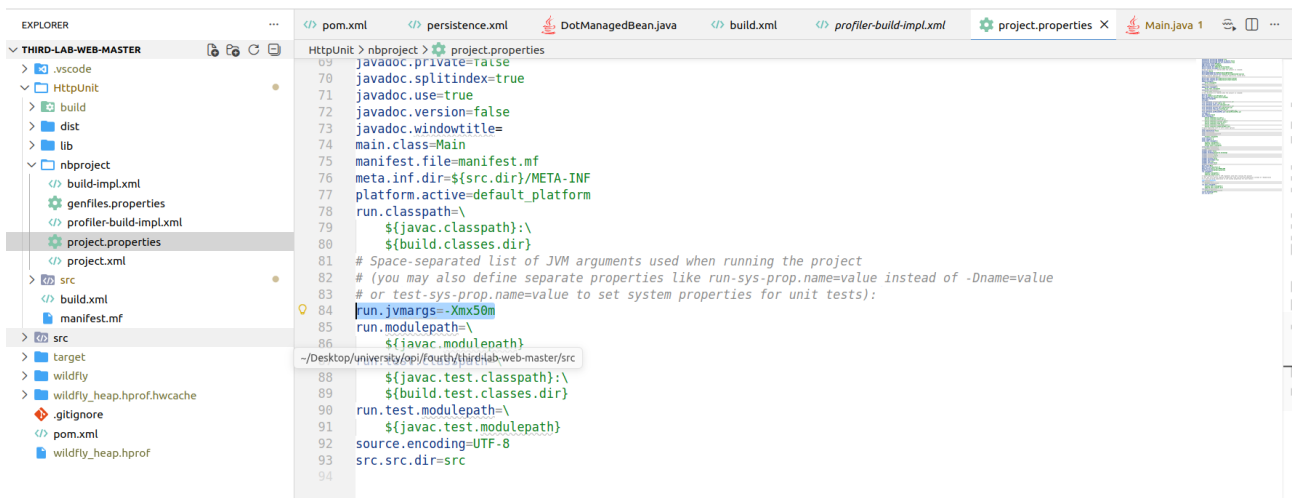


Рисунок 14. Добавление флага для увеличения доступного объема памяти

В мониторинге памяти видна высокая скачкообразная нагрузка на Garbage Collector — рисунок 15.

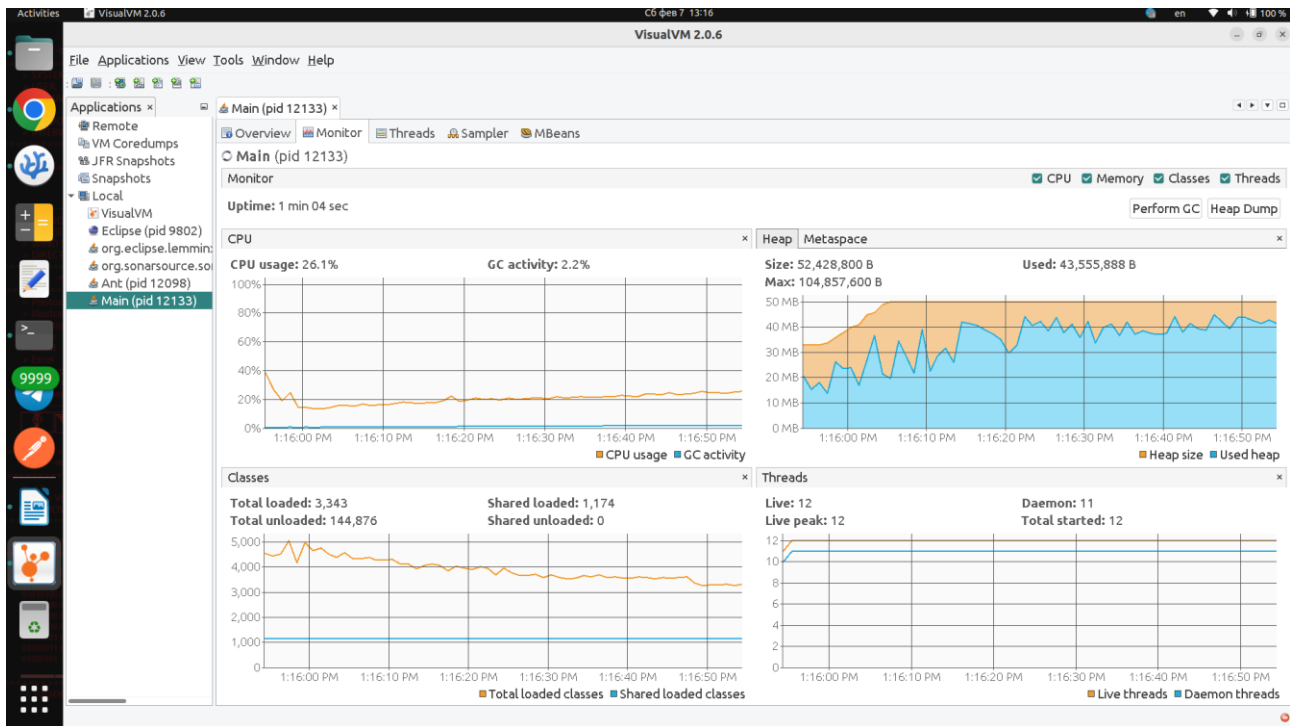


Рисунок 15. Просмотр использования памяти.

При помощи Heap Dump найдем объекты, которые занимают большую часть памяти.

При помощи команды `jps` находим id запущенной программы (Main):

```
jps -l
```

При помощи следующей команды создаем heap dump:

```
map -dump:live,format=b,file=wildfly_heap.hprof 57473
```

Выполнение команд можно увидеть на рисунке 16.

```
sus@sus-computer: ~/Desktop/university/opi/fourth/third-lab-web-...
sus@sus-computer:~/Desktop/university/opi/fourth/third-lab-web-master$ jps -l
57473 Main
58625 jdk.jcmd/sun.tools.jps.Jps
31189 /home/sus/.vscode-oss/extensions/redhat.java-1.52.0-universal/server/plugins/org.eclipse.equinox.launcher_1.7.100.v20251111-0406.jar
31399 /home/sus/.vscode-oss/extensions/sonarsource.sonarlint-vscode-4.42.0-universal/server/sonarlint-ls.jar
31753 org.eclipse.lemminx.XMLServerLauncher
57438 org.apache.tools.ant.launch.Launcher
sus@sus-computer:~/Desktop/university/opi/fourth/third-lab-web-master$ jmap -dump:live,format=b,file=wildfly_heap.hprof 57473
Dumping heap to /home/sus/Desktop/university/opi/fourth/third-lab-web-master/wildfly_heap.hprof ...
Heap dump file created [55935878 bytes in 0,071 secs]
sus@sus-computer:~/Desktop/university/opi/fourth/third-lab-web-master$
```

Рисунок 16. Создание Heap Dump.

Анализ использования памяти в VisualVM представлен на рисунках 17, 18 и 19.

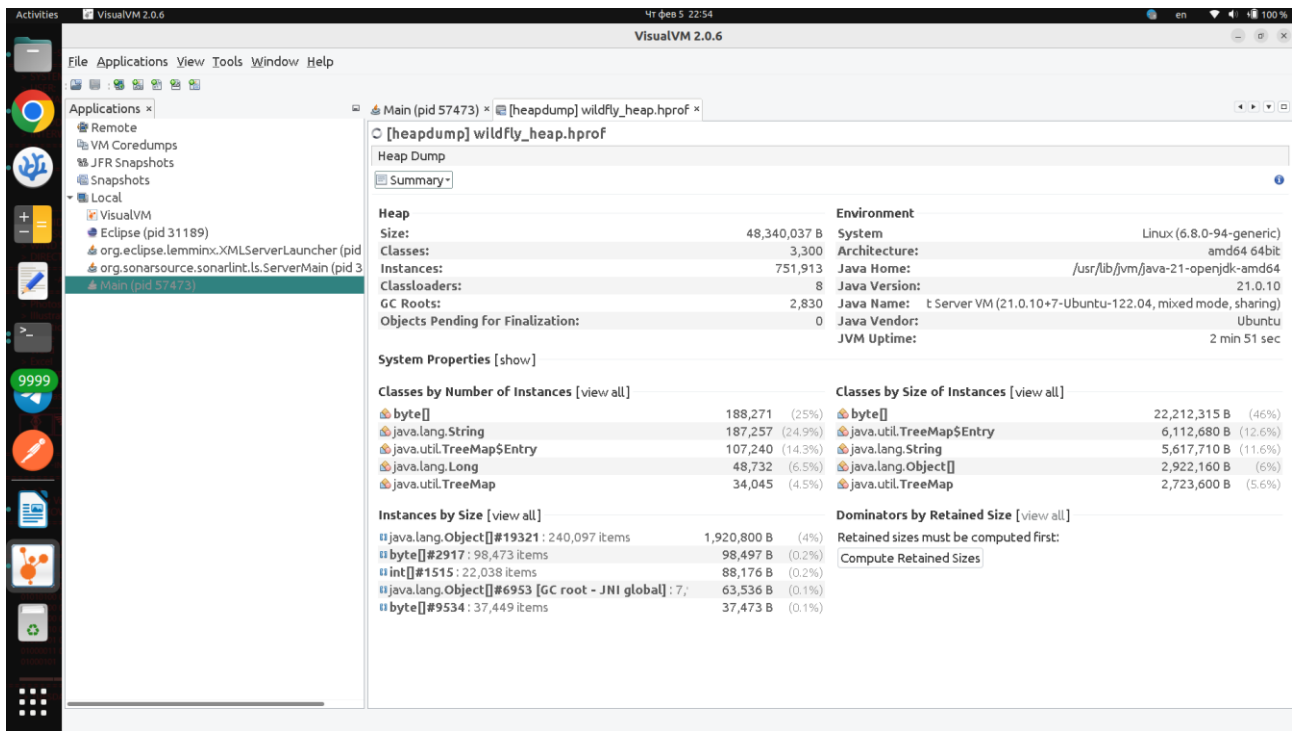


Рисунок 17. Summary Heap Dump

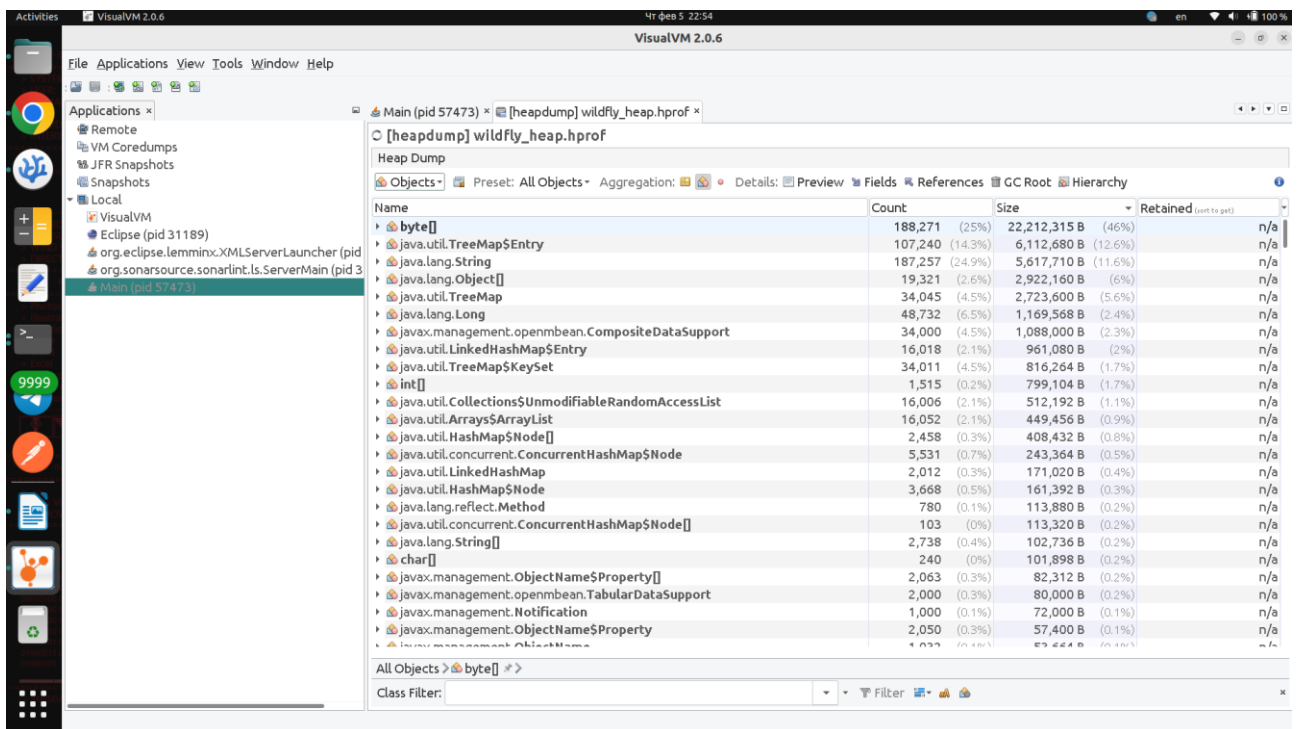


Рисунок 18. Список объектов в памяти

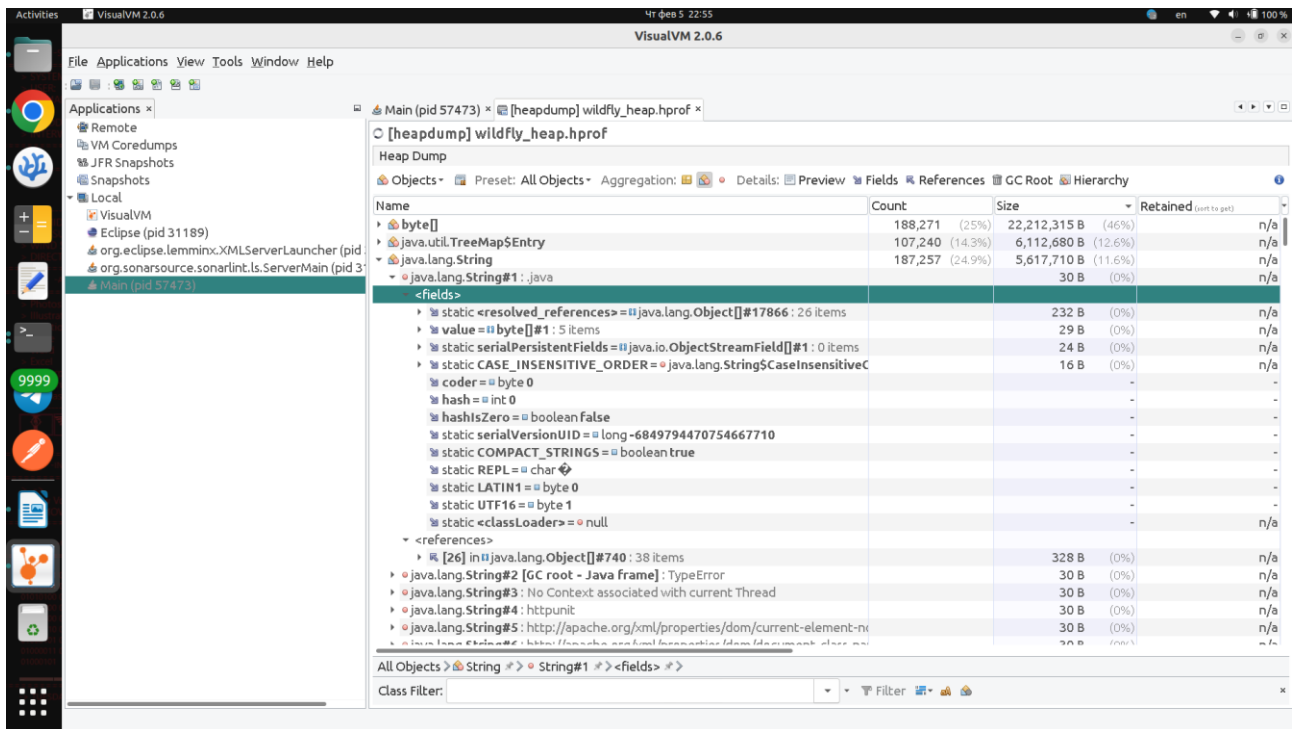


Рисунок 19. Просмотр подробностей об объектах

На представленных рисунках видно, что при работе приложения на каждый запрос создаются экземпляры строк и массивы `byte[]`, которые используются по всему приложению, но при этом остаются в памяти.

Garbage Collector работает корректно, однако несмотря на это размер кучи неуклонно растет. Из-за описанных выше проблем через некоторое время мы получаем ошибку `OutOfMemoryError`.

Исследовав самые крупные объекты в куче, находим проблемный массив объектов — рисунок 20.

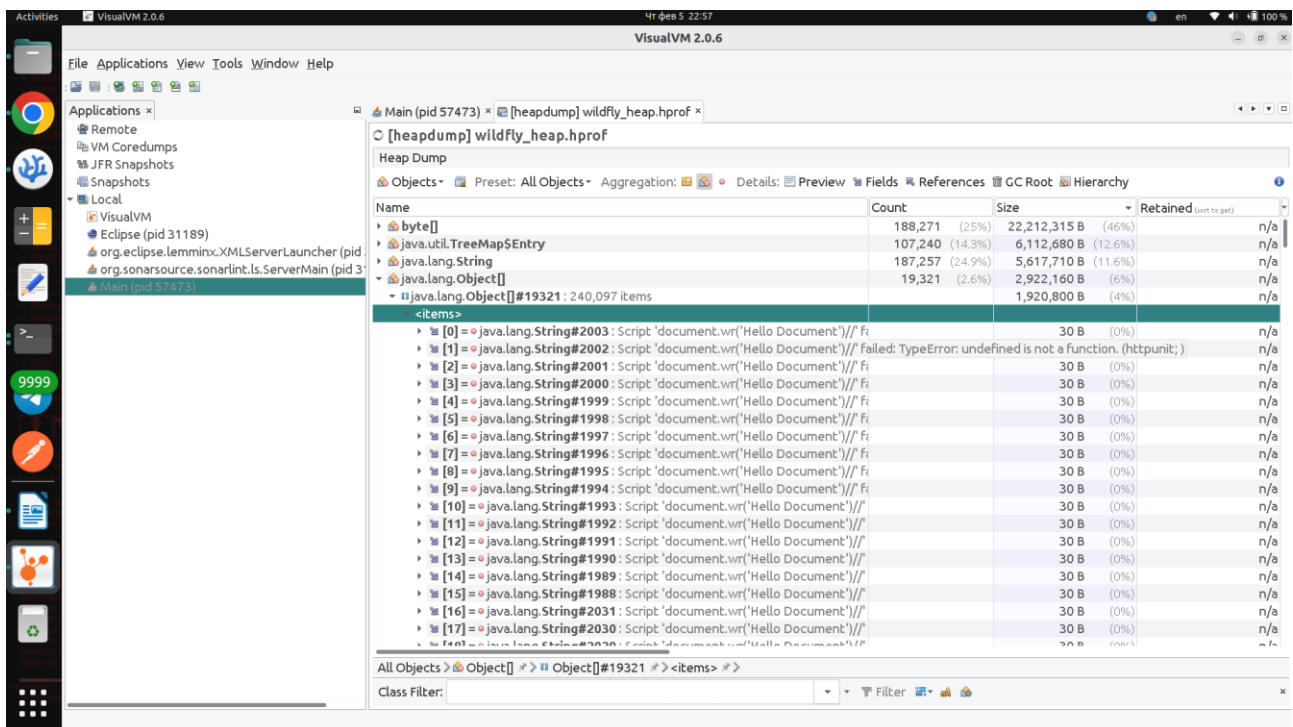


Рисунок 20. Проблемный массив объектов

Этот массив самый крупный по размеру и в нем дублируются строки, поэтому перейдем для анализа в класс в котором он содержится — рисунок 21.

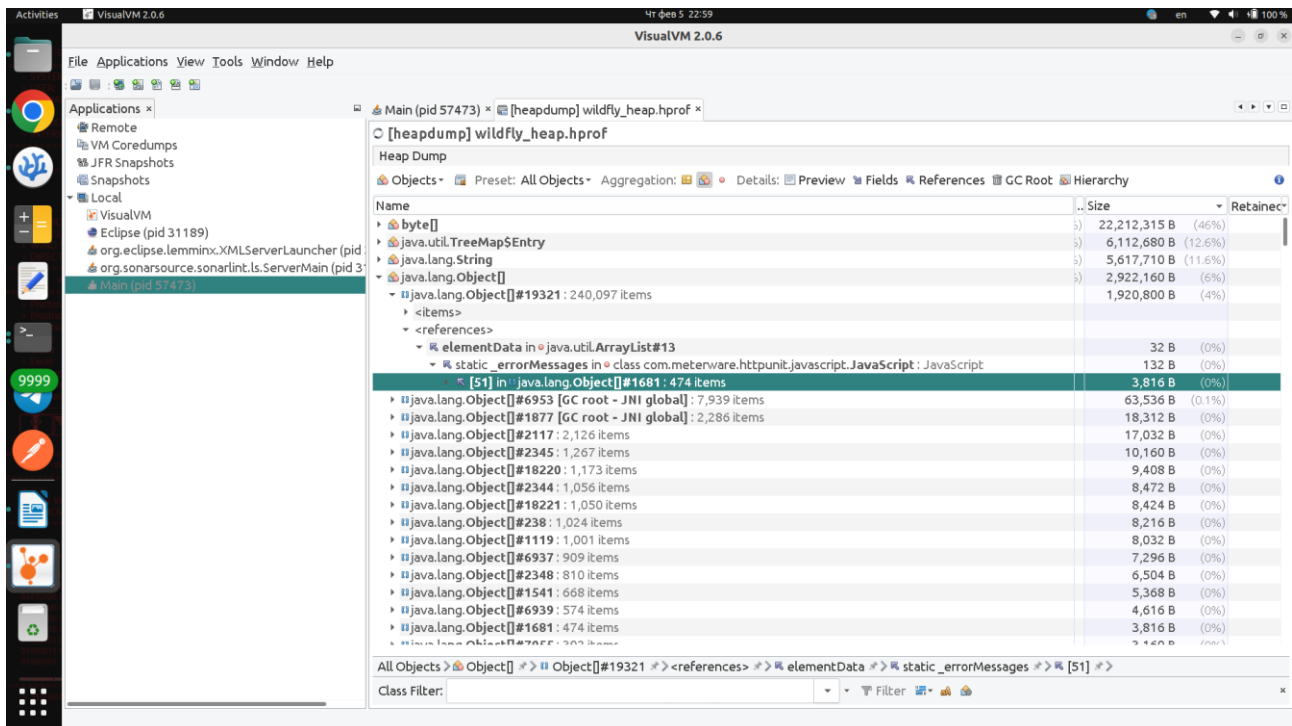


Рисунок 21. Находим проблемный класс

Получается это класс JavaScript в пакете com.meterware.httpunit.javascript. Проанализируем этот класс, в нем находится проблемное статическое поле `_errorMessages`, которое содержит эту переполненную структуру данных — рисунок 22.

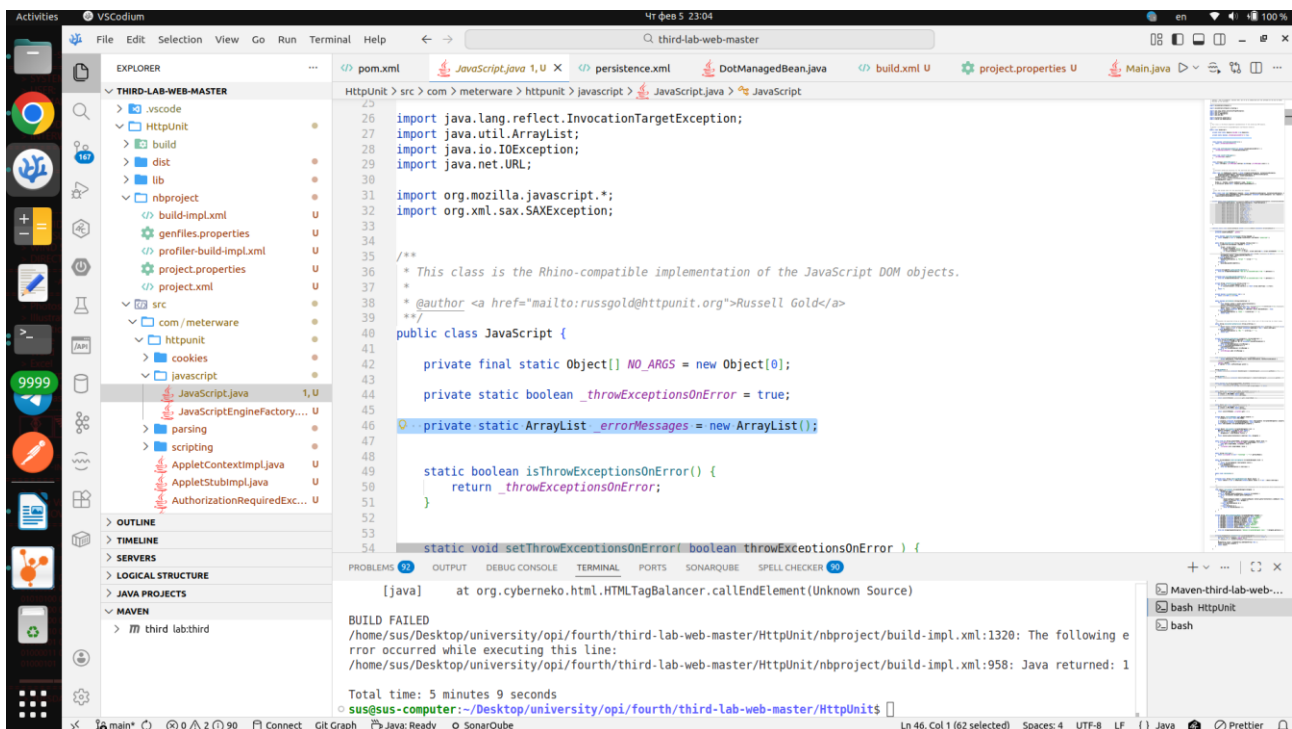


Рисунок 22. Проблемное поле в классе

Находим место где наполняется список — рисунок 23.

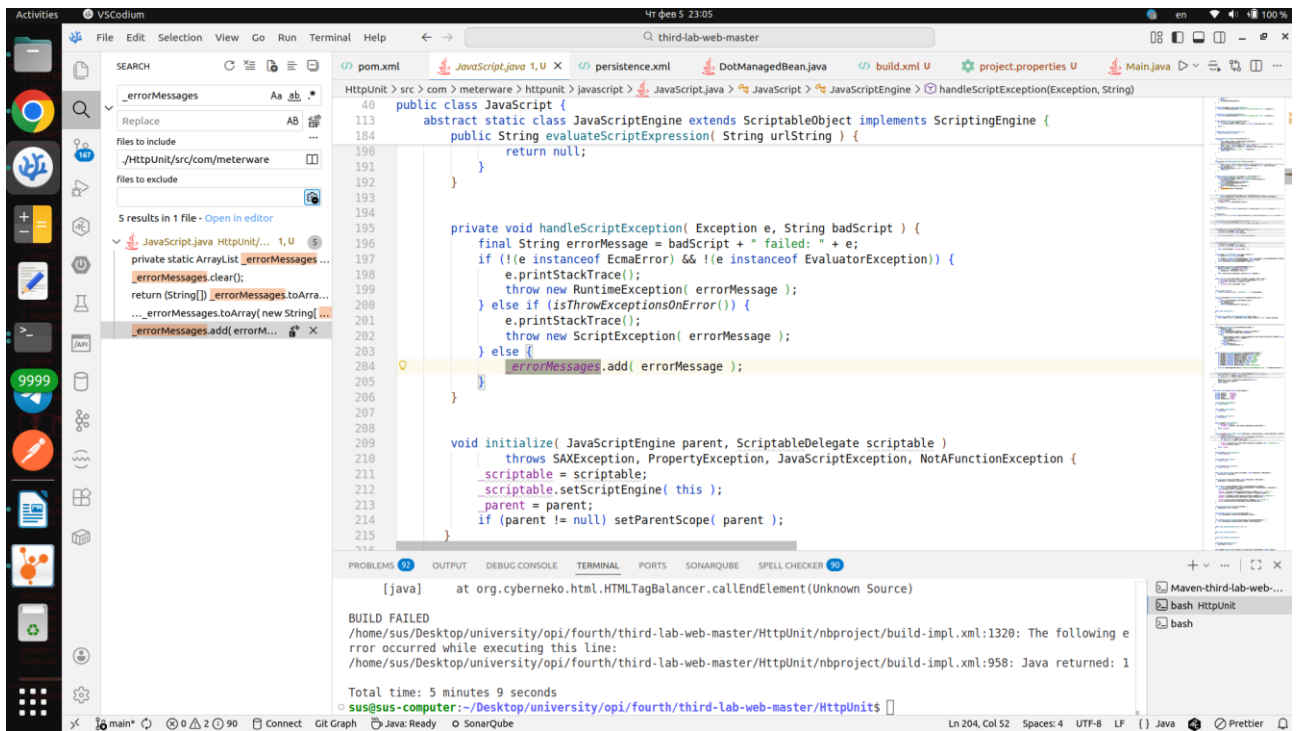


Рисунок 23. Место наполнения проблемного списка

И также есть метод очистки списка — рисунок 24. Он используется только в методе обертке класса `HttpUnitOptions`, при этом метод обертка нигде не используется — рисунок 25.

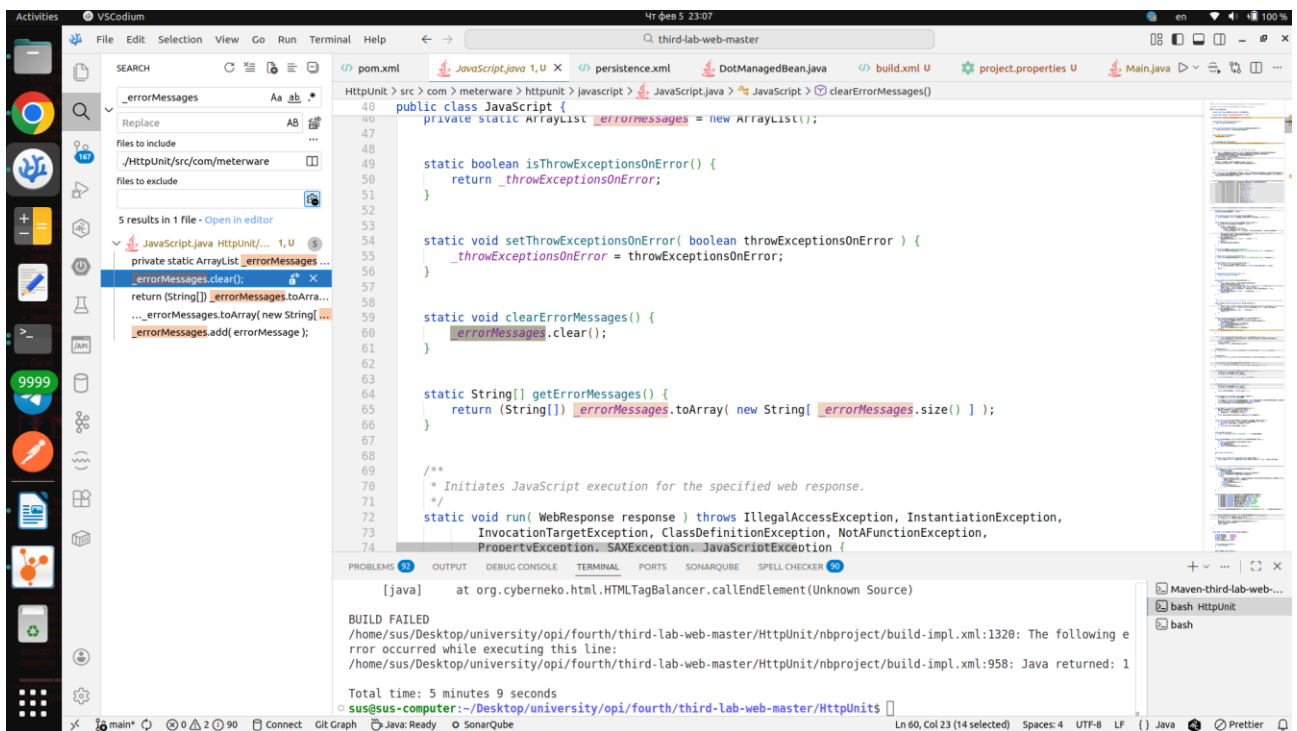


Рисунок 24. Метод очистки списка

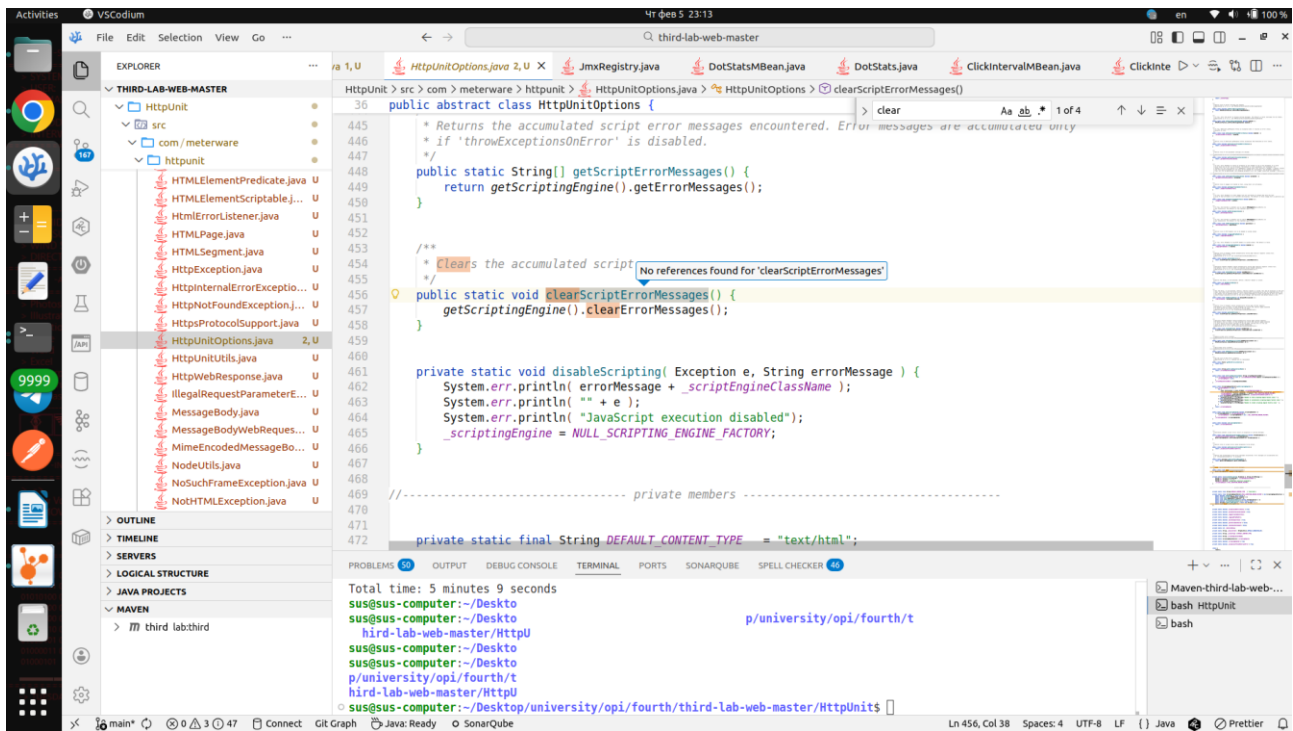


Рисунок 25. Метод обертка для метода очистки списка

То есть список заполняется но при этом нигде не очищается. Исправим эту ошибку, добавив очистку после выполнения каждого запроса — рисунок 26.

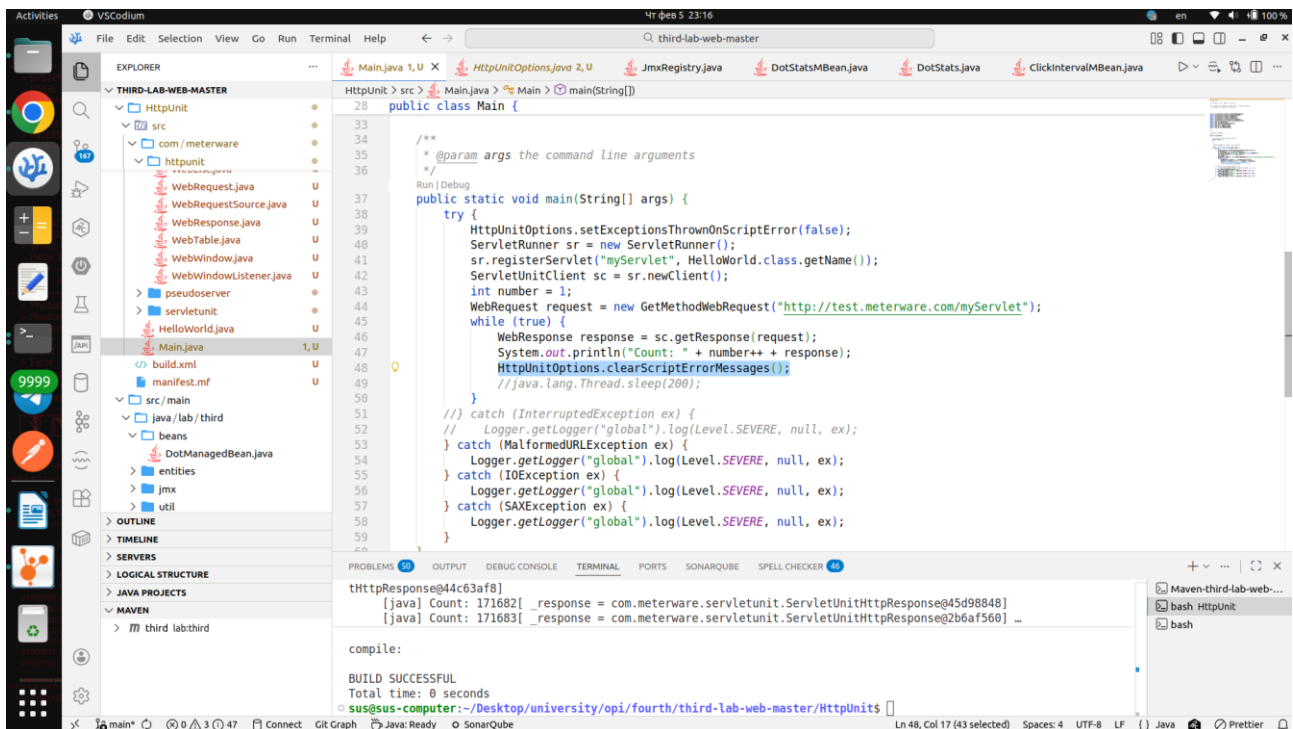


Рисунок 26. Исправление ошибки утечки памяти

Запустим после исправления приложение. Как видим на рисунках 27 и 28 теперь приложение работает стабильно.

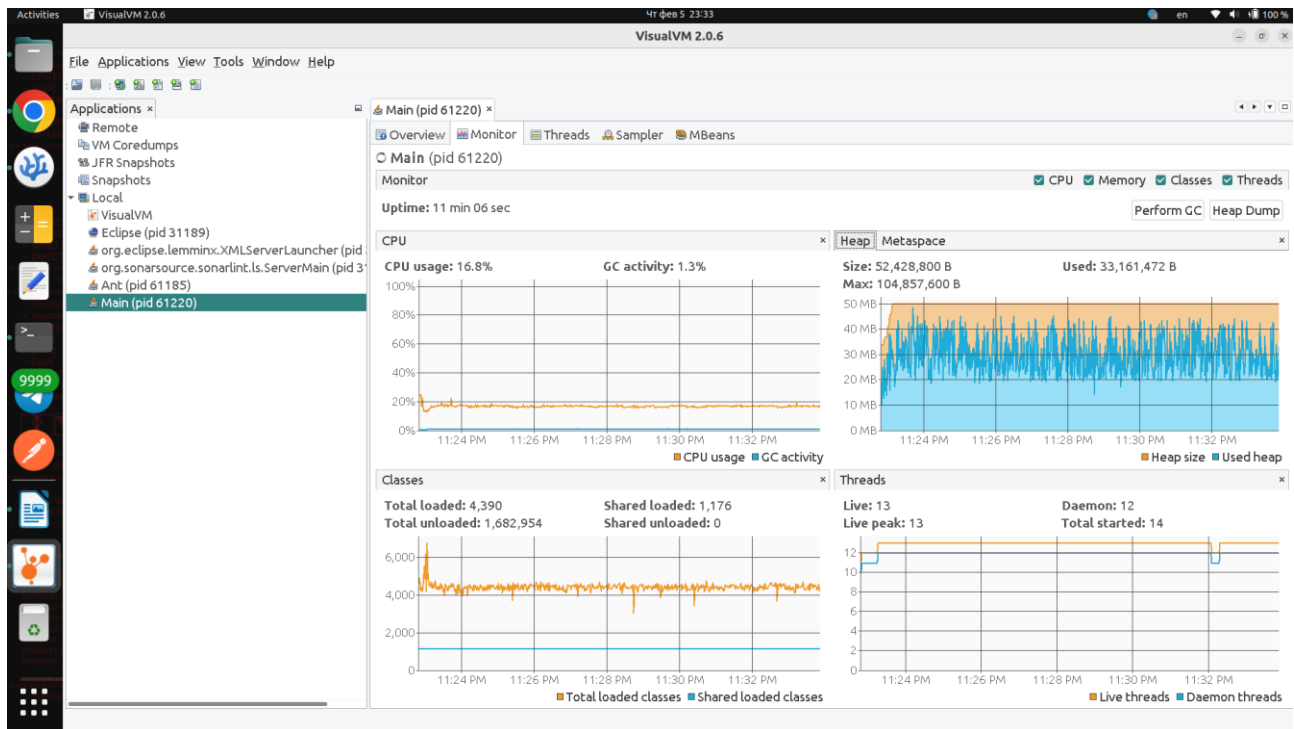


Рисунок 27. Стабильная работа приложения

Рисунок 28. Вывод в терминал при работе приложения

Выводы по работе

В рамках данной лабораторной работы я освоил практику разработки и интеграции MBeans для мониторинга веб-приложений. Для анализа производительности использовал инструменты JConsole и VisualVM, с помощью которых проводил мониторинг метрик, профилирование CPU и исследование работы сборщика мусора (GC). На основе полученных данных, таких как графики потребления памяти и отчеты heap dump, я научился выявлять узкие места, диагностировать утечки памяти и применять соответствующие оптимизации для их устранения.